

#17 · 失敗補償

[優化] 技術代號 A-P1-4 · 約 3-4 天

一句話：失敗任務不會清理

為什麼要修

一連串步驟其中一步失敗，前面已完成步驟結果留著（產生孤立資產、半成品垃圾）。

現在怎樣

shared/agent-plan-schema.ts 已有 compensationAction 與 rollbackPolicy 欄位，但 orbTaskChainRunner 沒有實際執行路徑。

問題在哪

影片合成、LoRA 訓練等高成本步驟失敗時無法乾淨退場，資料庫留半成品 + 已扣點數無法退款。

要改什麼

新增 compensation chain 執行邏輯（LIFO）+ compensating state + audit log 記錄。失敗的 compensation 本身不再遞迴補償，只記錄留人工介入。

涉及檔案

shared/orb-task-state-machine.ts （加 compensating state）

server/services/orbTaskChainRunner.ts （initializeCompensationContext / executeCompensationStep / runCompensationChain）

shared/agent-plan-schema.ts （compensationAction 文件強化）

Claude Code 提示詞 (複製貼上)

把下面這段話完整複製，貼到 Claude Code (或 /claude 對話)。Claude Code 會依此完成此項修復。完成後跑下方驗證步驟確認。

請依照計畫附錄 B.2 實作 A-P1-4 Compensation / Rollback 鏈：

1. 修改 shared/orb-task-state-machine.ts，在 ORB_TASK_STATES 加 "compensating" state：

```
```typescript
export const ORB_TASK_STATES = [
 "idle", "planning", "awaiting_approval", "approved",
 "executing", "paused", "recovering",
 "compensating", // ← 新增
 "completed", "failed", "cancelled", "blocked",
] as const;

export interface OrbAgentTaskStep {
 // 既有 + 新增：
 rollbackStatus?: "pending" | "completed" | "failed" | "skipped";
 compensationRetryCount?: number;
}

export const DEFAULT_COMPENSATION_RETRY_BUDGET = 1; // 補償失敗不再遞迴
```
```

2. 在 server/services/orbTaskChainRunner.ts 新增三個函式：

```
```typescript
export interface CompensationStep {
 originalStepId: string;
 compensationAction: unknown;
 executedAt: number;
 completedAt?: number;
 failedAt?: number;
 failureReason?: string;
}

export function initializeCompensationContext(task: OrbAgentTask, failedStepId: string) {
 const completed = task.steps
 .filter((s) => s.status === "completed" && s.id !== failedStepId)
 .sort((a, b) => b.id.localeCompare(a.id)); // LIFO
 return {

```

```

taskId: task.taskId,
failedAtStepId: failedStepId,
compensationQueue: completed
.filter((s) => s.compensationAction)
.map((s) => ({ originalStepId: s.id, compensationAction: s.compensationAction, executedAt: Date.now()
})),
startedAt: Date.now(),
};
}

```

```

export async function runCompensationChain(ctx, tools, updateTask) {
 await updateTask({ status: "compensating" });
 for (const step of ctx.compensationQueue) {
 try {
 await executeOrbToolCall(step.compensationAction, tools, { timeoutMs: 60_000 });
 step.completedAt = Date.now();
 appendOrbAgentTaskAuditEvent({ taskId: ctx.taskId, type: "step.rollback_completed", message:
`${step.originalStepId} rollback OK` });
 } catch (err) {
 step.failedAt = Date.now();
 step.failureReason = String(err);
 appendOrbAgentTaskAuditEvent({ taskId: ctx.taskId, type: "step.rollback_failed", message:
`${step.originalStepId} rollback failed: ${err}` });
 // 不遞迴 — 補償失敗只記錄
 }
 }
 const allOk = ctx.compensationQueue.every((s) => s.completedAt);
 await updateTask({ status: allOk ? "cancelled" : "failed" });
}
...

```

3. 在 orbTaskChainRunner 主 step loop 內，當任何 step status === "failed" 且 task.policy.rollbackPolicy === "auto-on-failure" 時：

```

```typescript
const ctx = initializeCompensationContext(task, failedStep.id);
await runCompensationChain(ctx, tools, (patch) => updateOrbAgentTask(task.taskId, patch));
...

```

4. 寫 vitest 測試：

- 三步 plan，第二步刻意 throw → 第一步的 compensation 應被執行、第三步 skip

- Compensation 本身 throw → audit log 有 step.rollback_failed、task 狀態為 failed (非 cancelled)
- rollbackPolicy: "manual" 時不自動執行

不要做的事：不要遞迴補償（補償失敗不再補償）。不要動 plan schema（既有欄位已備）。不要做 partial rollback（要嘛全部、要嘛失敗，沒中間態）。

如何驗證做完了

建一個三步 plan（生圖 → 加旁白 → 合成影片），第三步 throw → 看 audit

log：step.rollback_completed 兩條（第二、第一步）→ task 狀態 cancelled。模擬第二步 compensation throw → step.rollback_failed → task 狀態 failed。

預估耗時

3-4 天